

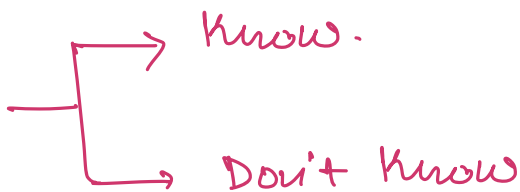
Design Splitwise.

⇒ We get the set of requirements from the interviewer in the form of a doc

Steps.

1. Overview
2. Clarifying requirements.
3. Class diagram.
4. Schema Design
5. Code.

SPLITWISE ⇒ Expense Sharing/Tracking App.

Overview. 
→ Know.
→ Don't Know

A } Total Expense: 2000 ₹
B }
C } Paid By: (A)
Everyone's share:

A: 500 ₹

B: 800 ₹

C: 700 ₹

$\Rightarrow$ A has paid 1500 ₹ extra

$\Rightarrow$ B has paid 800 ₹ lesser

$\Rightarrow$ C has paid 700 ₹ lesser

$B \rightarrow A : 800 ₹$

$C \rightarrow A : 700 ₹$

$\Rightarrow$ Expense : A, B, C, D

$\rightarrow$ Total amount : 5000 ₹

$\rightarrow$ Paid By : A : 2000 ₹

B : 3000 ₹

$\rightarrow$ Had To pay :

A : 1000

B : 2000

C : 500

D : 1500

(Share of every user)

$\Rightarrow$ A has paid 1000 ₹ extra.

B has paid 1000 ₹ extra.

C has paid 500 ₹ lesser.

D has paid 1500 ₹ lesser.

Total paid Extra $\equiv$ Total paid lesser.

User who has paid Extra should get money back from users who has paid lesser.

C $\xrightarrow{500\text{₹}}$ A

D $\xrightarrow{500\text{₹}}$ A

D $\xrightarrow{1000\text{₹}}$ B

OR

D $\xrightarrow{1000\text{₹}}$ A

C $\xrightarrow{500}$ B

D $\xrightarrow{500}$ B

After Settle Up, Nobody should've paid extra & nobody should've paid lesser.

Expense. $\Rightarrow$ Who paid for this expense.

$\rightarrow$ who paid $\Rightarrow$ Map < User, Amount >

$\rightarrow$ hadToPay $\Rightarrow$ Map < User, Amount >

$\rightarrow$ Share of each user in the expense.

$\rightarrow$ group $\Rightarrow$ Null | Not Null.

User who has paid lesser should pay to user who has paid Extra.

(+) $\Rightarrow$ Paid Extra

(-) $\Rightarrow$ Paid lesser

A, B, C, D.

+100
Ⓐ

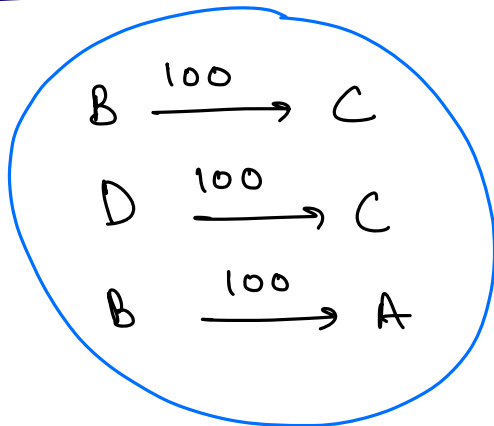
-200
Ⓑ

+200
Ⓒ

-100
Ⓓ

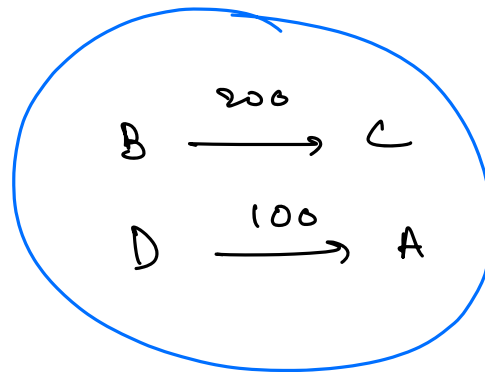
Settle Up.

①



3 transactions

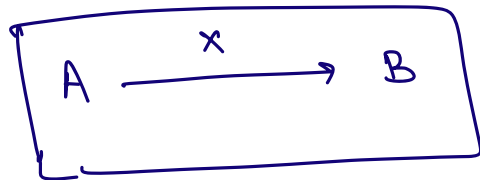
②



2 transactions

SettleUp.

⇒ There are N number of expenses that we want to settleup, We need to return the list of transactions required to settleup.



⇒ who $\xrightarrow{\text{amount}}$ whom

list(Txn) Settleup (list<Expense> expenses) {

⇒ Expenses.

└─ who paid: [— — —]

└─ who had to pay: [— — — —]

A, B, C, D.

$$\Sigma \text{ who paid} = \Sigma \text{ hadToPay}$$

E1:
who paid $\Rightarrow$ A: 1000, B: 1000
hadToPay $\Rightarrow$ A: 500, B: 500, C: 500, D: 500

E2:
who paid $\Rightarrow$ A: 3000
hadToPay $\Rightarrow$ A: 1000, B: 200, C: 800, D: 1000

E3:
who paid $\Rightarrow$ C: 500, D: 800
hadToPay $\Rightarrow$ A: 500, B: 100, C: 200, D: 500

E4:
who paid $\Rightarrow$ D: 1000
hadToPay $\Rightarrow$ A: 250, B: 250, C: 250, D: 250

$\Rightarrow$ find out for each user, the final amount who has paid extra & who has paid lesser.

for each user:

extra-amount = 0

for each expense:

extra-amount += whoPaid[user]

extra-amount -= hadToPay[user]

$$A: (1000 - 500) + (3000 - 1000) + (0 - 500) + (0 - 250)$$

$$: \textcircled{+} \underline{\underline{1750}}$$

$$B: \underline{-50}$$

$$C: \underline{-1250}$$

$$D: \underline{\underline{-450}}$$

$$\underline{\underline{\Sigma = 0}}$$

$$+1750$$

$\textcircled{A}$

$$\textcircled{B}^{-50}$$

$$-450$$

$\textcircled{D}$

$$\textcircled{C}^{-1250}$$

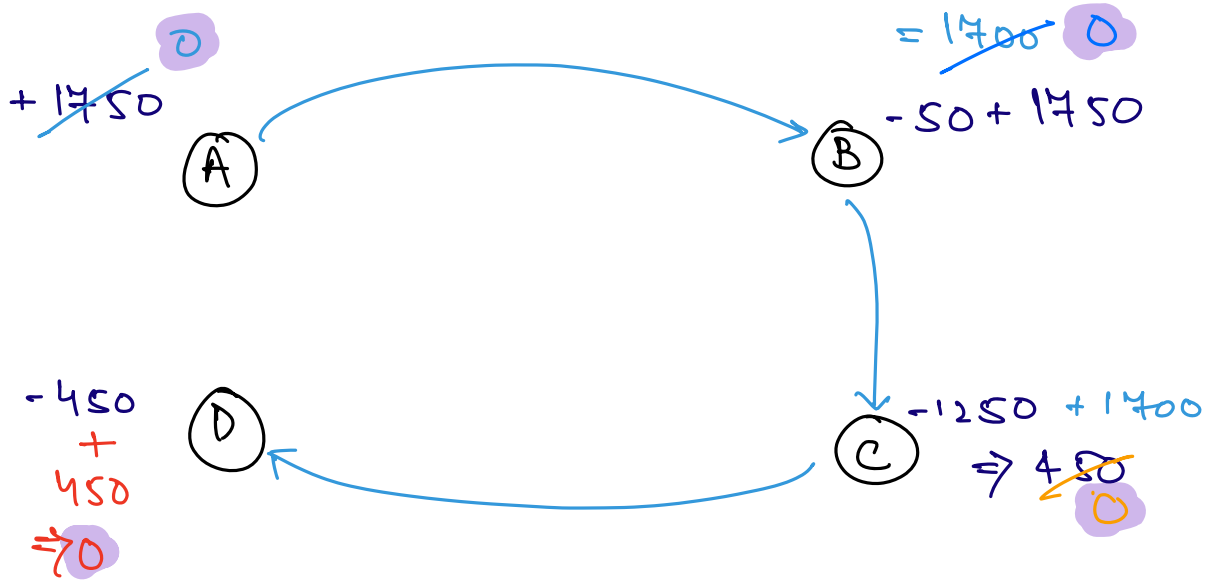
Minimize the # of transactions.



NP hard problem.

(only exponential solⁿ)

Solⁿ 1



$B \longrightarrow A : 1750$

$C \longrightarrow B : 1700$

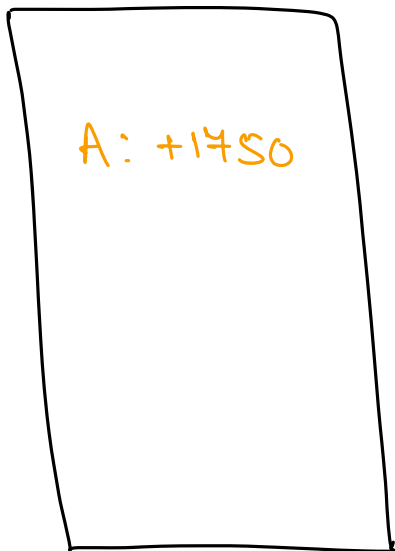
$D \longrightarrow C : 450$

$\Rightarrow$ Solⁿ is correct, but Not Practical.

① Divide the users in 2 buckets.

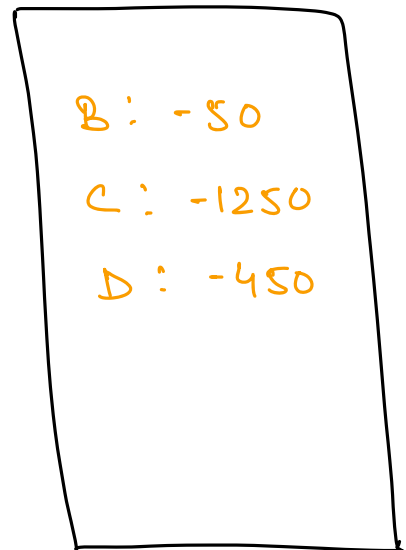
Users who has
paid extra

G-1



Users who has
paid lesser

G-2



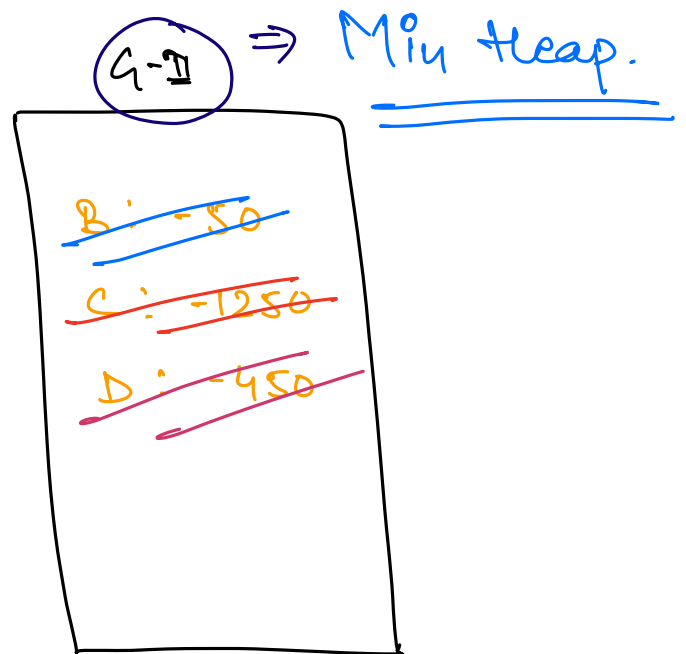
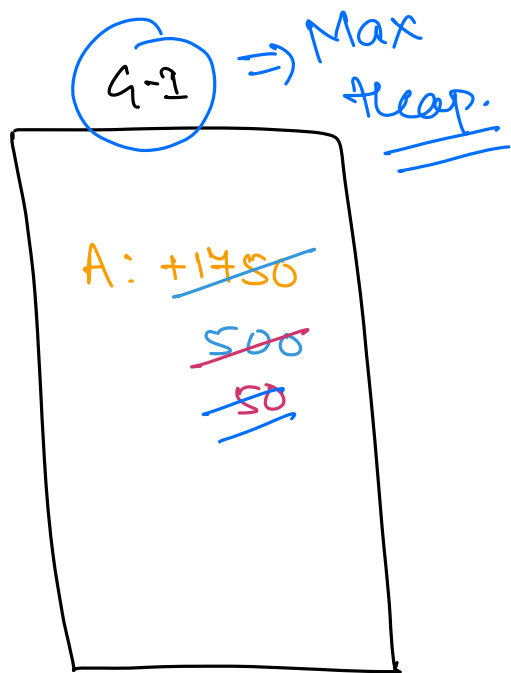
Group 1 users should pay to Group 2 users.

→ Get the Min from Group ① ⇒ ~~X~~

→ Get the Max from Group ② ⇒ Y

⇒ ~~X~~ → Y

⇒ Atleast one of ~~X~~ or Y will get settled up.



①

A +1450 500	C -1250 0	$\Rightarrow \min(A, C)$ $= 1250$
------------------------------	----------------------------	--------------------------------------

C $\xrightarrow{1250}$ A

②

A: 500 50	D: -450 0	$\Rightarrow \min(D, A)$ $\Rightarrow \underline{\underline{450}}$
-------------------------	-------------------------	---

D $\xrightarrow{450}$ A

③

A: 50 0	B: -50 0	$\min(A, B)$
-----------------------	------------------------	--------------

B $\xrightarrow{50}$ A

⇒ Revise Heap DS.

⇒ Class Diagram

⇒ Schema Design.

} HW.